

Sports MVP Rewrite Specification and Milestone Plan

Project Goal

Rebuild the prediction, selection, scoring, validation, and simulation layers into a clean, testable system that supports:

- NBA, NCAAM, NHL
 - Moneyline, spread, total for all three leagues
 - The existing operational flow:
 - Generate Snapshots
 - Finalize Closes
 - Score
 - Validate
 - A 45-day on-demand simulation workflow
 - A premium, consistent product foundation suitable for subscription growth
-

Non-Negotiable Rules

- No more patching the legacy scoring logic.
 - New scoring and prediction work goes into brand-new files.
 - Snapshot generation, finalize closes, scoring persistence, and CLV support remain in the platform architecture.
 - All leagues must support all three market types.
 - The rewrite must be validated before any production cutover.
-

Phase Overview

Phase 0 — Recovery and Freeze

Goal: Stabilize the app and stop touching the old scoring layer beyond recovery.

Deliverables - [] Restore app to a syntactically valid state - [] Confirm API boots successfully - [] Confirm current prediction endpoints respond - [] Freeze legacy scoring logic for reference only - [] Document current pain points and known failures

Exit Criteria - API and web app run without syntax/runtime corruption - Legacy logic is treated as reference, not active design direction

Phase 1 — Rewrite Specification

Goal: Finalize the architecture, formulas, contracts, metrics, and simulator design before implementation.

Deliverables - ☐ Prediction architecture defined - ☐ Selection/scoring architecture defined - ☐ Data contracts defined - ☐ Validation metrics defined - ☐ 45-day simulator design defined - ☐ File map defined - ☐ Implementation order approved

Exit Criteria - The new system is fully specified and approved before coding begins

Phase 2 — Core Scoring Engine

Goal: Build the shared pricing, EV, ranking, and tiering engine in new files.

Deliverables - ☐ Fair market probability utility - ☐ EV calculator - ☐ Edge calculator - ☐ Rank engine - ☐ Tier assignment engine - ☐ Shared scoring config

Exit Criteria - Shared scoring layer works for all leagues and markets with one consistent contract

Phase 3 — Prediction Models by League and Market

Goal: Build separate model modules for each league/market combination.

Deliverables - ☐ NBA moneyline model - ☐ NBA spread model - ☐ NBA total model - ☐ NCAAAM moneyline model - ☐ NCAAAM spread model - ☐ NCAAAM total model - ☐ NHL moneyline model - ☐ NHL spread model - ☐ NHL total model

Exit Criteria - Nine model modules return outputs in one shared format

Phase 4 — Calibration Layer

Goal: Calibrate model outputs using out-of-sample historical data.

Deliverables - ☐ Sigmoid calibration support - ☐ Isotonic calibration support - ☐ Per-model calibration selection logic - ☐ Reliability-bucket reporting - ☐ Calibration metadata storage

Exit Criteria - Each model has a chosen calibration method based on validation results

Phase 5 — Scoring Pipeline Integration

Goal: Connect the new model and scoring engines into the existing operational workflow.

Deliverables - ☐ Generate Snapshots compatibility confirmed - ☐ Finalize Closes compatibility confirmed - ☐ Score compatibility confirmed - ☐ Validate compatibility confirmed - ☐ CLV preserved in the pipeline

Exit Criteria - New prediction/selection logic runs without removing snapshot/finalize/CLV behavior

Phase 6 — 45-Day Simulator

Goal: Build an on-demand simulator to evaluate new model versions safely.

Deliverables - ☐ `simulate45Days.js` design implemented - ☐ Inputs: start date, days, leagues, markets, version - ☐ Historical replay from available snapshots/data - ☐ Outputs: ROI, hit rate, EV, CLV, Brier, log loss, drawdown, picks/day - ☐ Per-league and per-market breakdowns

Exit Criteria - The simulator can compare candidate scoring/model versions before deployment

Phase 7 — Validation and Cutover

Goal: Decide if the new engine is good enough to replace legacy selection/scoring.

Deliverables - ☐ 45-day validation run completed - ☐ Rolling 14/30/45-day review completed - ☐ League/market-level breakdown reviewed - ☐ Calibration performance reviewed - ☐ CLV performance reviewed - ☐ Production cutover recommendation made

Exit Criteria - New engine wins on validation and is approved for production switch

Architecture Specification

1. System Separation

The new system is split into two independent layers:

A. Prediction Layer

Responsible for estimating probabilities or fair distributions for each market.

B. Selection and Scoring Layer

Responsible for comparing model outputs to market prices, computing edge and EV, ranking bets, assigning tiers, and preparing picks.

This separation prevents model probabilities and business logic from being tangled together.

2. Supported Markets

Every league supports every market:

- Moneyline
- Spread
- Total

Supported leagues: - NBA - NCAAM - NHL

This creates nine model paths total.

3. Prediction Design

Moneyline Models

Output direct win probability for each side.

Spread Models

Output expected margin distribution, then convert to cover probability at the offered spread.

Total Models

Output expected total distribution, then convert to over/under probability at the offered total.

4. Calibration Design

Each model output is calibrated out of sample.

Supported calibration methods: - Sigmoid - Isotonic

Per-model calibration metadata: - calibration method - calibration version - training window - validation scores

5. Market Pricing Layer

All market prices are converted into fair probabilities.

Inputs

- publish odds

- close odds
- side/market metadata

Outputs

- implied market probability
- fair market probability after vig removal

For two-sided markets, normalize implied probabilities so the two sides sum to 1.

6. Core Bet Calculations

A. Edge

```
edge = model_prob_calibrated - market_prob_fair
```

B. Expected Value

Using decimal odds `d` and calibrated probability `p`:

```
EV = p * (d - 1) - (1 - p)
```

C. CLV

CLV remains in the validation pipeline and compares publish price vs finalized close price.

7. Ranking Design

One ranking formula for all leagues and markets.

Candidate Fields Used

- normalized EV
- normalized edge
- calibration confidence
- market liquidity confidence

Proposed Rank Score

```
rank_score = 0.50 * normalized_ev + 0.25 * normalized_edge + 0.15 *
calibration_confidence + 0.10 * market_liquidity_confidence
```

Tiering

- Tier A

- Tier B
- Tier C
- PASS

Tiering is derived from score bands, not scattered hard-coded gates.

8. Risk Controls

Risk controls are allowed, but they must live in one place and be explicit.

Examples: - stale or missing market price - insufficient confidence in calibration bucket - low-liquidity or low-quality market data - duplicate correlated picks on the same game if later required

Risk controls are not allowed to be hidden across multiple files.

File Map

```
apps/api/src/prediction/  
  nbaMoneylineModel.js  
  nbaSpreadModel.js  
  nbaTotalModel.js  
  ncaamMoneylineModel.js  
  ncaamSpreadModel.js  
  ncaamTotalModel.js  
  nhlMoneylineModel.js  
  nhlSpreadModel.js  
  nhlTotalModel.js  
  
apps/api/src/scoring/  
  marketProb.js  
  calibration.js  
  expectedValue.js  
  rankBets.js  
  assignTiers.js  
  scorePicks.js  
  validatePicks.js  
  
apps/api/src/simulation/  
  simulate45Days.js  
  
apps/api/src/config/  
  scoringModelConfig.js
```

Data Contracts

1. Candidate Contract

Each candidate bet must return this shape:

```
{
  "league": "nba|ncaam|nhl",
  "market_type": "moneyline|spread|total",
  "side": "home|away|over|under",
  "game_key": "string",
  "event_start": "ISO timestamp",
  "publish_odds": 0,
  "publish_line": 0,
  "model_prob_raw": 0.0,
  "model_prob_calibrated": 0.0,
  "market_prob_fair": 0.0,
  "edge": 0.0,
  "ev": 0.0,
  "rank_score": 0.0,
  "tier": "A|B|C|PASS",
  "calibration_method": "sigmoid|isotonic|none",
  "calibration_version": "string",
  "market_quality": 0.0,
  "selection_reason": "string|null"
}
```

2. Scored Pick Contract

```
{
  "date": "YYYY-MM-DD",
  "league": "nba|ncaam|nhl",
  "game_key": "string",
  "market": "moneyline|spread|total",
  "pick": "home|away|over|under|PASS",
  "result": "win|loss|push|null",
  "publish_odds": 0,
  "publish_line": 0,
  "close_odds": 0,
  "close_line": 0,
  "model_prob_raw": 0.0,
  "model_prob_calibrated": 0.0,
  "market_prob_fair": 0.0,
}
```

```
"edge": 0.0,  
"ev": 0.0,  
"rank_score": 0.0,  
"tier": "A|B|C|PASS",  
"clv_line_delta": 0.0,  
"clv_implied_delta": 0.0,  
"meta": {}  
}
```

Validation Metrics

The new engine is judged by these categories:

1. Calibration

- Brier score
- Log loss
- Reliability buckets
- Predicted vs actual by bucket

2. Market Performance

- CLV hit rate
- Average CLV by league and market
- Average fair edge captured

3. Betting Performance

- ROI
- Win rate
- Units won/lost
- Maximum drawdown

4. Product Coverage

- Picks per day
- Picks by league
- Picks by market
- PASS rate

5. Stability

- Rolling 14-day metrics
 - Rolling 30-day metrics
 - Rolling 45-day metrics
-

45-Day Simulator Design

Purpose

Provide an on-demand test harness for new model/scoring versions before production cutover.

Script

```
apps/api/src/simulation/simulate45Days.js
```

Inputs

- `start_date`
- `days` (default 45)
- `leagues` (default nba,nhl,ncaam)
- `markets` (default moneyline,spread,total)
- `model_version`
- `scoring_version`
- `calibration_version`

Process

1. Load historical schedule and available market snapshots for each date
2. Generate model predictions using only data available as of that date
3. Calibrate probabilities using the selected calibration version
4. Build candidate bets for all leagues and markets
5. Convert market prices to fair probabilities
6. Compute edge, EV, rank score, and tier
7. Persist or simulate publish selections
8. Finalize closes from available closing data
9. Score outcomes
10. Compute validation outputs

Outputs

- total picks
- picks/day
- ROI
- win rate
- units won/lost
- average EV
- average edge
- average CLV
- CLV hit rate
- Brier score

- log loss
 - maximum drawdown
 - by-league breakdown
 - by-market breakdown
 - date-by-date timeline
-

Cutover Rules

The new engine should not replace the legacy engine until:

- it is syntactically stable
 - it completes the full pipeline
 - 45-day simulation results are reviewed
 - rolling validation is acceptable
 - calibration is materially better than the legacy version
 - CLV behavior is preserved
-

Immediate Next Milestones

Milestone 1 — Recovery Complete

- ☐ Recover valid `predict.js`
- ☐ Confirm API syntax passes
- ☐ Confirm app runs again

Milestone 2 — Rewrite Spec Approved

- ☐ Approve this specification
- ☐ Lock formulas and file map
- ☐ Lock simulator requirements

Milestone 3 — Greenfield Build Starts

- ☐ Create new scoring files
 - ☐ Create new config file
 - ☐ Implement shared utilities first
-

Notes for Implementation Discipline

- All new work must go into new files first.
- No regex-based code rewrites.
- No hidden market toggles.
- No spread/total/moneyline disable switches.

- No route-level scoring logic duplication.
- Every formula must live in one place.
- Every milestone must be checked off before moving to the next phase.